



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD: INFORMÁTICA Y ELECTRÓNICA
ESCUELA DE INGENIERÍA EN SISTEMAS
CARRERA: SOFTWARE

GUÍA DE LABORATORIO DE APLICACIONES INFORMÁTICAS II

PRÁCTICA No. 1

1. DATOS GENERALES:

NOMBRE DEL ESTUDIANTE

STIVEN APOLO

CÓDIGO DEL ESTUDIANTE

7243

FECHA DE REALIZACIÓN:

2026/07/08

FECHA DE ENTREGA:

2026/07/08

2. OBJETIVO(S):

2.1. GENERAL

Aplicar la técnica de ingeniería inversa sobre la aplicación ClinicFlow, con el propósito de comprender su estructura, componentes, modelo de datos y procesos principales como base para el mantenimiento de software.

2.2. ESPECÍFICOS

- Identificar los módulos, roles, entidades y flujos funcionales principales de la aplicación ClinicFlow.
- Analizar de forma estática la estructura del frontend, backend, rutas, controladores, modelos y servicios auxiliares.
- Construir el diagrama de clases, diagrama de componentes, diseño de base de datos y diagramas de secuencia a partir del código fuente revisado.

- Documentar el proceso de ingeniería inversa sin ejecutar la aplicación, priorizando la comprensión técnica para actividades posteriores de mantenimiento.

3. METODOLOGÍA

La práctica se desarrolló mediante análisis estático del sistema, por lo que no se ejecutó la aplicación ni se realizaron pruebas funcionales en tiempo real. La ingeniería inversa se aplicó revisando la estructura del repositorio, los archivos fuente y las relaciones entre módulos, con el fin de reconstruir modelos de diseño que faciliten la comprensión del software.

N.º	Actividad	Descripción
1	Selección del aplicativo	Se seleccionó ClinicFlow por corresponder a una aplicación web relacionada con gestión clínica y citas médicas.
2	Revisión de estructura	Se identificaron carpetas principales como frontend, backend, rutas, controladores, modelos, configuración y servicios externos.
3	Análisis de entidades	Se revisaron los modelos asociados a usuarios, departamentos, doctores, citas y colas de atención.
4	Análisis de componentes	Se identificaron los bloques principales: Next.js, Express, MongoDB, Redis, Socket.IO, Cloudinary y servicio de correo.
5	Trazabilidad de procesos	Se relacionaron rutas, controladores y modelos para inferir los procesos implementados.
6	Construcción de diagramas	Se generaron espacios para insertar los diagramas resultantes de clases, componentes, base de datos y secuencias.

4. EQUIPOS Y MATERIALES:

- Computador
- Entorno integrado de desarrollo (IDE)
- Aula virtual
- Acceso a internet
- Repositorio de código fuente de ClinicFlow
- Herramienta para elaboración de diagramas UML
- Bibliografía relacionada con mantenimiento de software e ingeniería inversa

5. MARCO TEÓRICO:

5.1. Mantenimiento de software

El mantenimiento de software comprende las actividades realizadas sobre una aplicación después de su entrega o publicación, con el objetivo de corregir fallos, adaptarla a nuevos entornos, mejorar su calidad o prevenir problemas futuros. Para iniciar un proceso de

mantenimiento es necesario comprender previamente la estructura interna del sistema, sus componentes y sus procesos funcionales.

5.2. Ingeniería inversa

La ingeniería inversa es una técnica que permite analizar un sistema existente para recuperar información de diseño, arquitectura, procesos y datos a partir del código fuente o de artefactos disponibles. En esta práctica, la ingeniería inversa se utiliza como actividad inicial del mantenimiento, ya que permite comprender cómo está construido ClinicFlow antes de proponer o aplicar modificaciones.

5.3. Diagramas UML utilizados

UML permite representar diferentes vistas de un sistema de software. Para esta práctica se consideran el diagrama de clases, el diagrama de componentes y los diagramas de secuencia. Cada uno aporta una perspectiva distinta: estructura de entidades, organización de módulos e interacción temporal entre participantes.

5.4. Diseño de base de datos en aplicaciones MongoDB

En aplicaciones basadas en MongoDB, el diseño de base de datos se representa mediante colecciones, documentos embebidos y relaciones por referencia. En ClinicFlow se identifican colecciones como users, departments, doctors y appointments, además del uso de Redis como almacenamiento temporal y caché.

6. PROCEDIMIENTO:

6.1. Descarga y selección del aplicativo

Se seleccionó una aplicación referente al ámbito de gestión hospitalaria denominada ClinicFlow. El sistema permite analizar procesos comunes de una clínica, tales como registro de pacientes, autenticación, consulta de departamentos, visualización de doctores, reserva de citas, check-in en recepción y atención por parte del doctor.

La práctica se centró en comprender el sistema a partir del código fuente y su estructura, sin ejecutar la aplicación. Por tanto, los diagramas fueron obtenidos mediante revisión técnica de archivos, nombres de módulos, rutas, modelos y controladores.

6.2. Identificación general del sistema

Elemento	Descripción
Roles principales	admin, receptionist, doctor, patient
Frontend	Aplicación web desarrollada con Next.js y componentes reutilizables.
Backend	API desarrollada con Express, organizada en rutas, controladores, middlewares y modelos.
Base de datos	MongoDB con colecciones para usuarios, departamentos, doctores, citas y cola.

Caché y estado temporal	Redis para datos temporales, caché de consultas y estado de turnos.
Tiempo real	Socket.IO para notificaciones y actualización de turnos.
Servicios externos	Cloudinary para imágenes y proveedor de correo para OTP o recuperación de contraseña.

6.3. Análisis para el diagrama de clases

Para elaborar el diagrama de clases se revisaron las entidades principales del dominio. Se identificaron clases persistentes como Users, Departments, Doctors, Appointments y Queues. También se reconocieron objetos embebidos como image dentro de Users y schedule dentro de Doctors.

Clase / elemento	Interpretación obtenida por ingeniería inversa
Users	Representa a todas las personas del sistema y diferencia su función mediante el campo role.
Departments	Representa las especialidades o áreas médicas disponibles en la clínica.
Doctors	Representa el perfil médico asociado a un usuario y a un departamento.
Appointments	Representa las citas médicas, su estado, horario, QR y token de atención.
Queues	Representa una cola de atención secundaria; la cola operativa se puede derivar de Appointments y Redis.
Controladores	Se identificaron controladores para autenticación, administración, paciente, doctor y recepción.

6.4. Análisis para el diagrama de componentes

Para el diagrama de componentes se separó la aplicación en bloques funcionales. En el cliente se identificaron componentes visuales, hooks, store de Redux y consumo de API mediante Axios. En el servidor se identificaron rutas, controladores, middlewares, modelos, configuración de Socket.IO, conexión a MongoDB y conexión a Redis.

Componente	Función
Browser / Usuario	Punto de acceso del usuario al sistema web.
Frontend Next.js	Interfaz, páginas y componentes de la aplicación.
Redux Store	Estado global y peticiones asincrónicas desde el cliente.
API Express	Expone rutas REST por rol y coordina la lógica del backend.

MongoDB	Persistencia principal del sistema.
Redis	Caché y estado temporal.
Socket.IO	Actualización en tiempo real de turnos y estados.
Cloudinary / Correo	Servicios externos para imágenes y mensajes de verificación.

6.5. Análisis para el diseño de base de datos

El diseño de base de datos fue obtenido identificando los modelos del backend y sus relaciones. Como se trata de MongoDB, el modelo se interpreta mediante colecciones, referencias entre documentos y documentos embebidos.

Colección	Campos principales identificados
users	fullName, email, phoneNumber, password, image, role
departments	name, description
doctors	userId, departmentId, chamberNumber, consultationFee, slotDuration, schedule[]
appointments	patientId, doctorId, date, slotStart, slotEnd, status, qrHash, tokenNumber, isSkipped, checkedIn
queues	appointmentId, doctorId, date, tokenNumber, priority, status, checkedInAt, calledAt, completedAt

6.6. Análisis para los diagramas de secuencia

Los diagramas de secuencia se infirieron a partir de la relación entre páginas del frontend, rutas del backend, controladores, modelos y servicios auxiliares. Se priorizaron los procesos que representan el flujo principal de la aplicación clínica.

Proceso	Participantes identificados
Registro de paciente	Paciente, Frontend, AuthController, Redis, servicio de correo y MongoDB.
Login	Usuario, Frontend, AuthController, Redis, MongoDB y cookie httpOnly.
Consultar departamentos	Usuario, Frontend, UserController, Redis y MongoDB.
Ver doctores por departamento	Usuario, Frontend, UserController, Redis y MongoDB con populate.
Reservar cita médica	Paciente, Frontend, PatientController, Redis, MongoDB y generador QR.

Check-in en recepción	Recepcionista, Frontend, ReceptionistController, MongoDB, Redis y Socket.IO.
Dashboard del doctor	Doctor, Frontend, DoctorController, Redis y MongoDB.
Llamar siguiente paciente	Doctor, Frontend, DoctorController, MongoDB, Redis y Socket.IO.
Completar cita médica	Doctor, Frontend, DoctorController, MongoDB, Redis y Socket.IO.

6.7. Espacios para insertar los diagramas generados

En esta sección se dejan los espacios correspondientes para insertar los diagramas generados. Los nombres de archivo corresponden a los diagramas preparados para el envío de la práctica. Debido a que esta actividad se enfocó en el análisis y la ingeniería inversa, los espacios se dejan disponibles para colocar las imágenes finales dentro del documento.

Figura 1. Diagrama de clases

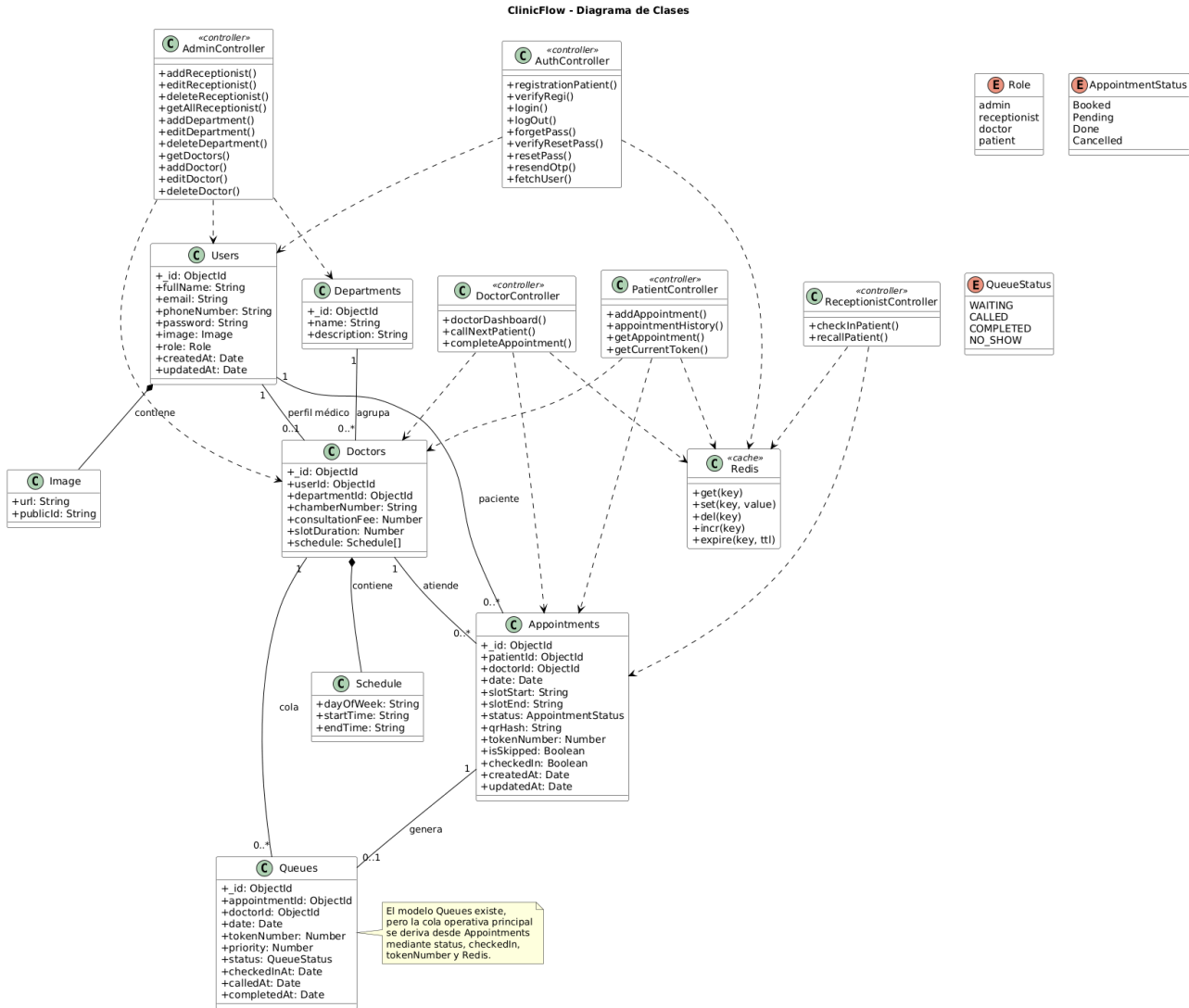


Figura 2. Diagrama de componentes

ClinicFlow - Diagrama de Componentes

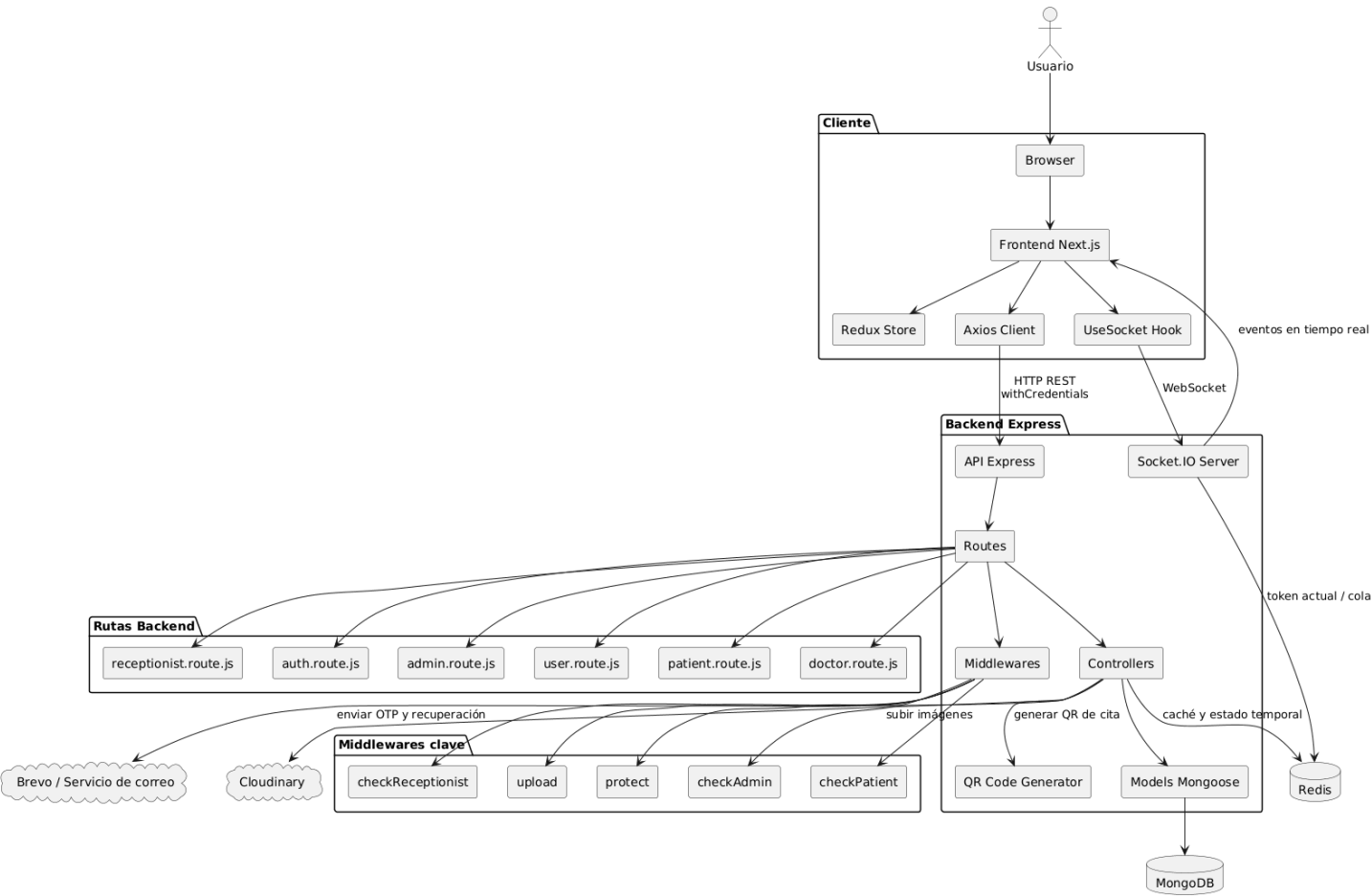


Figura 3. Diseño de base de datos

ClinicFlow - Diseño de Base de Datos MongoDB

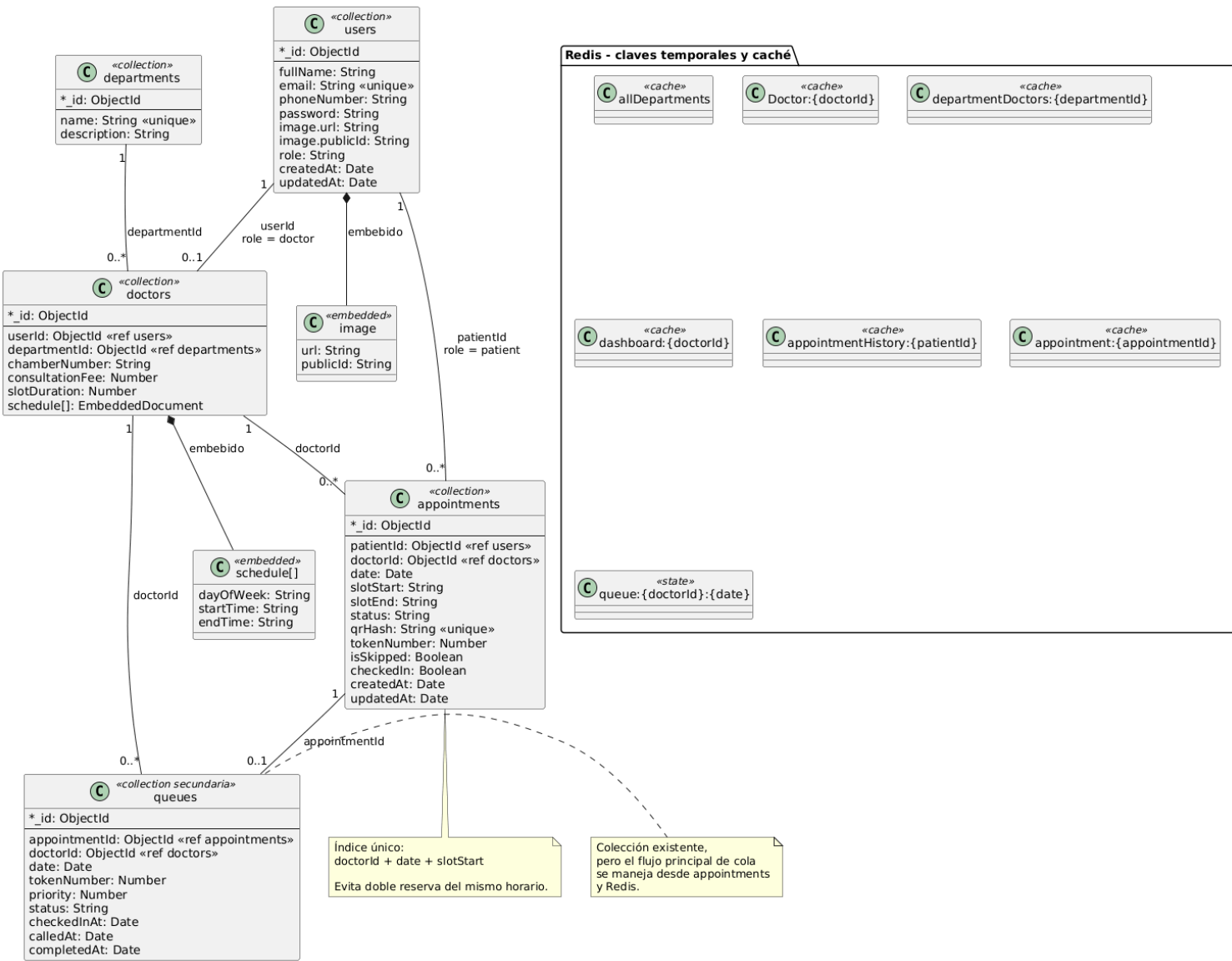


Figura 4. Diagrama de secuencia: registro de paciente

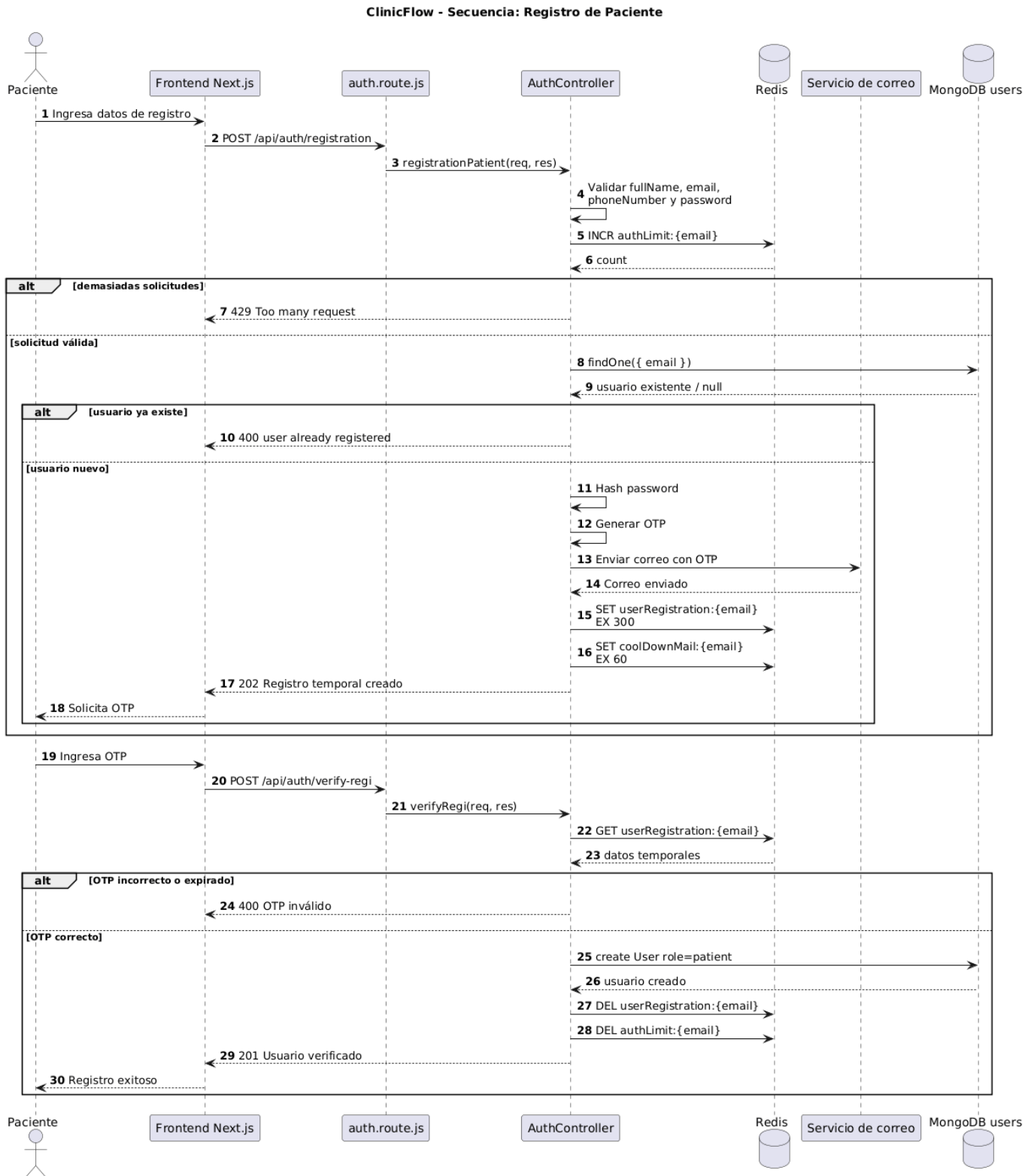


Figura 5. Diagrama de secuencia: login

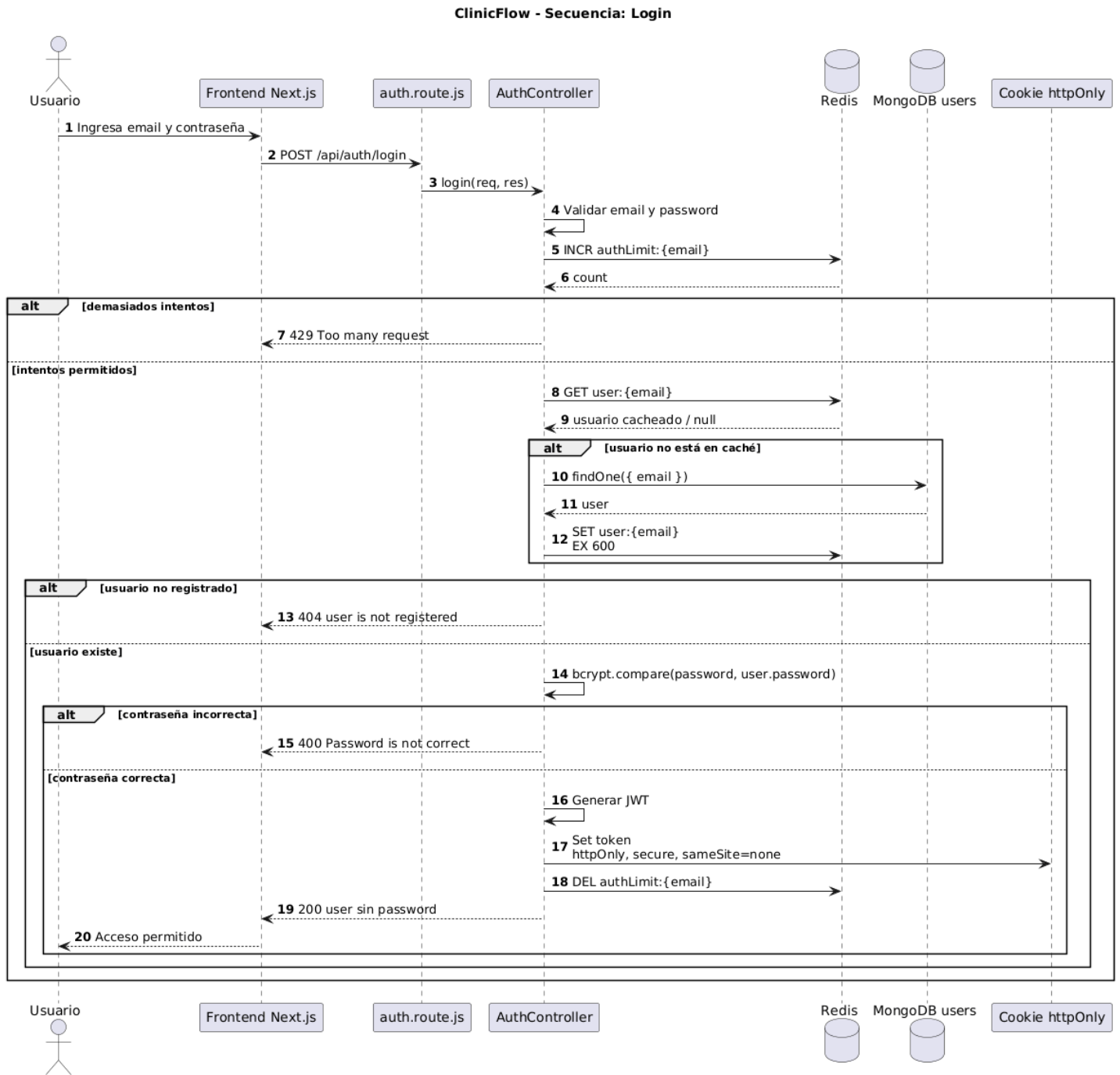


Figura 6. Diagrama de secuencia: consultar departamentos

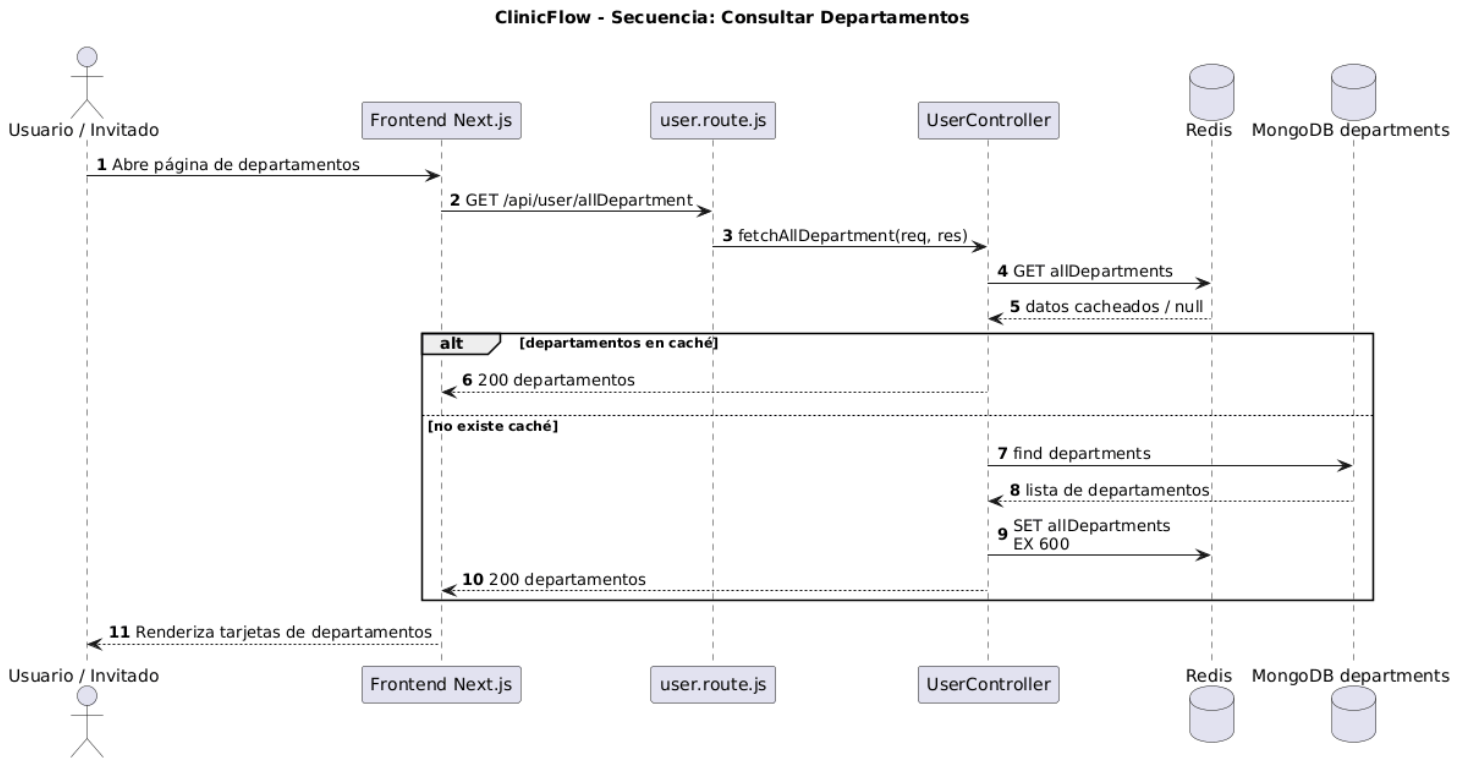


Figura 7. Diagrama de secuencia: ver doctores por departamentos

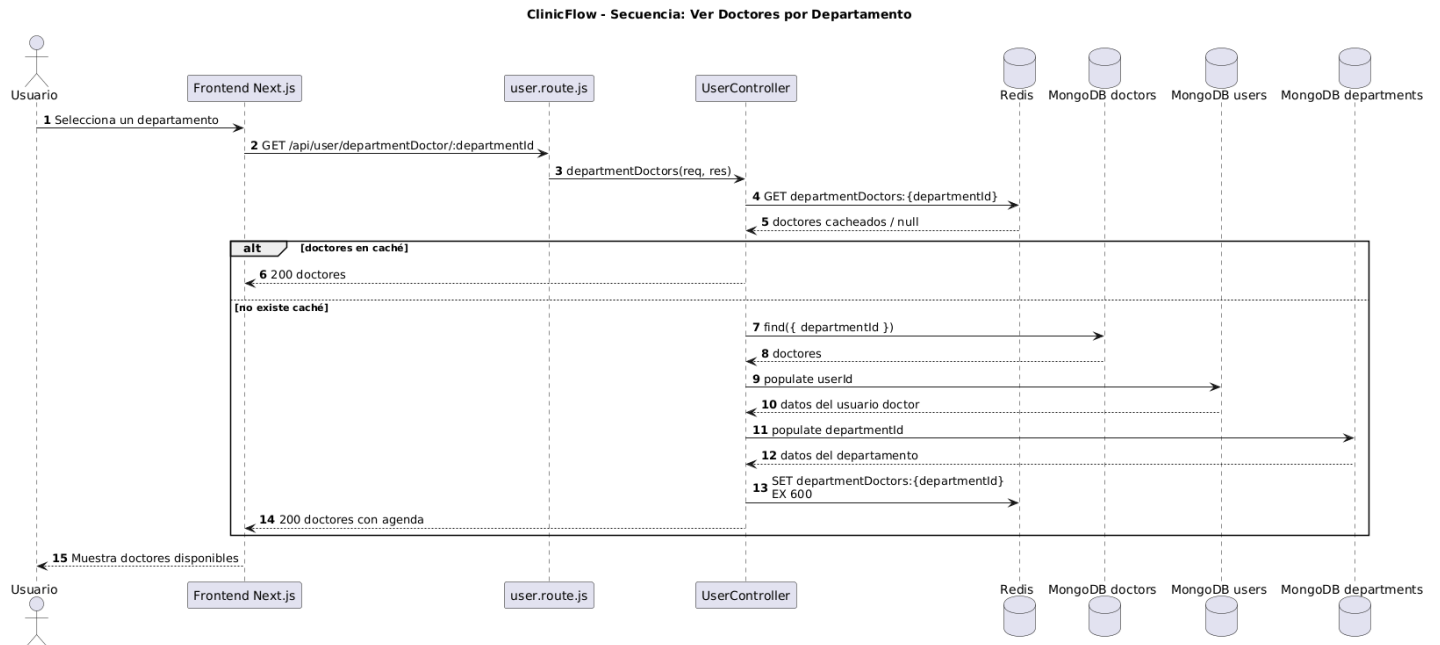


Figura 8. Diagrama de secuencia: reservar cita médica

ClinicFlow - Secuencia: Reservar Cita Médica

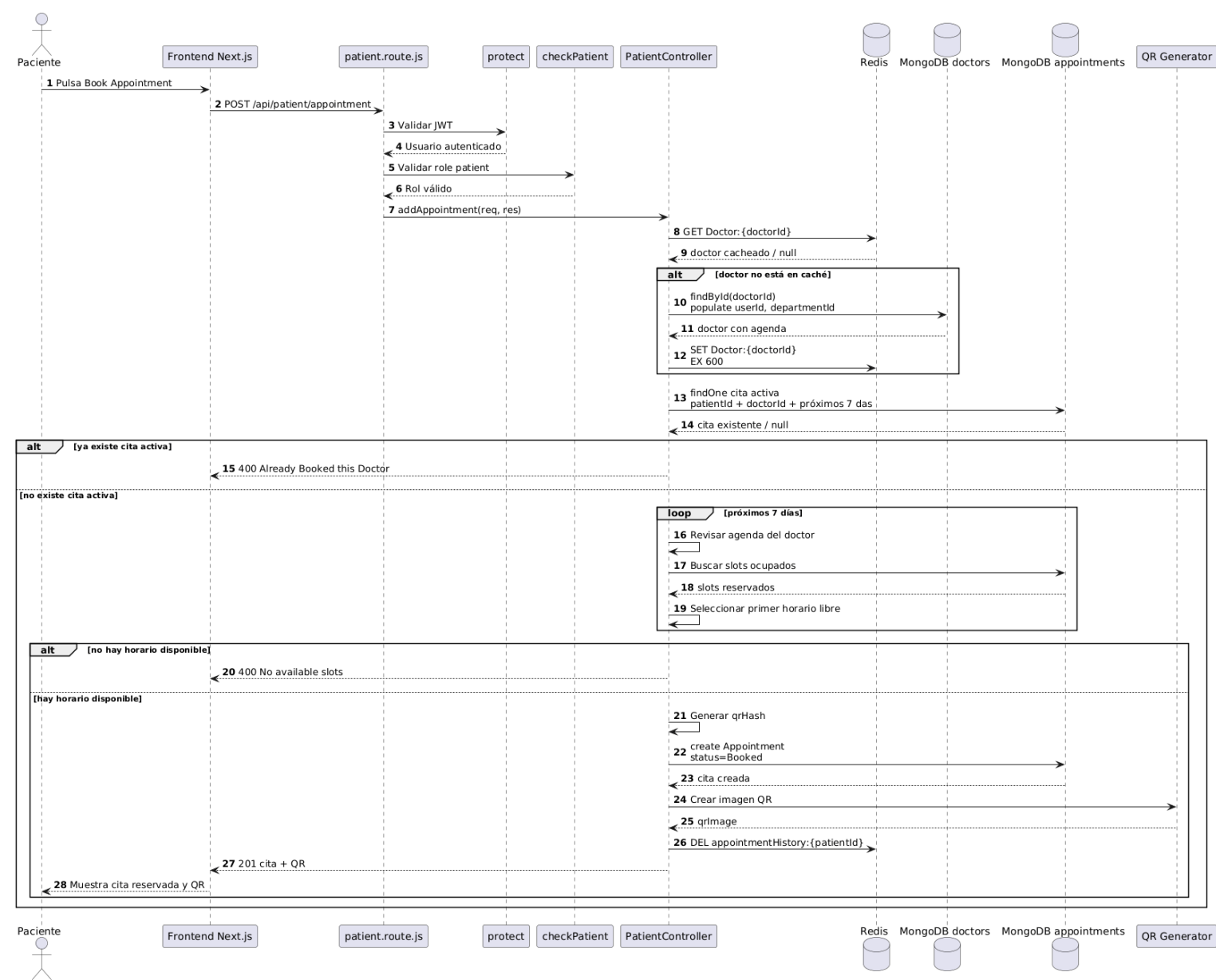


Figura 9. Diagrama de secuencia: check-in en recepción

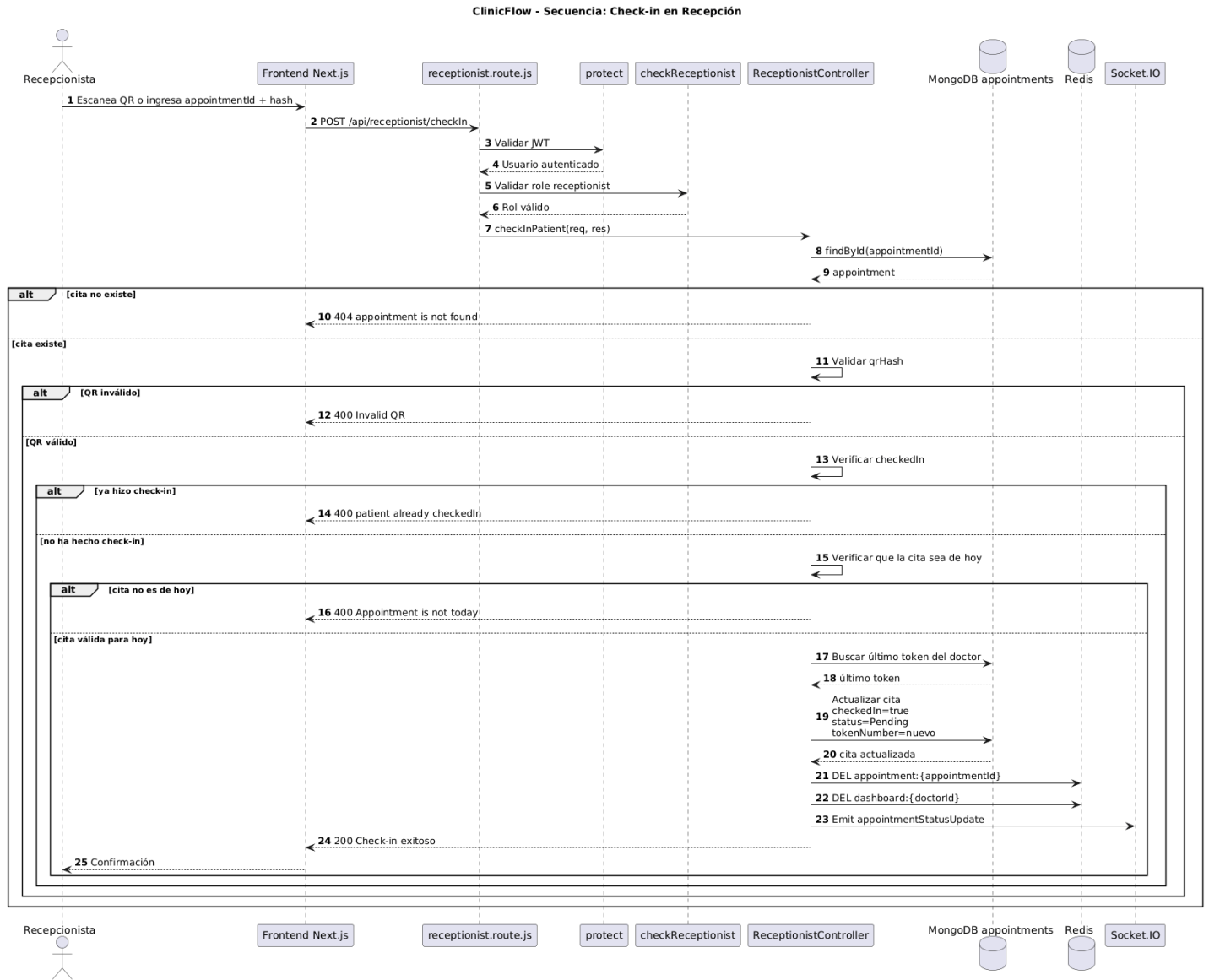


Figura 10. Diagrama de secuencia: dashboard del doctor

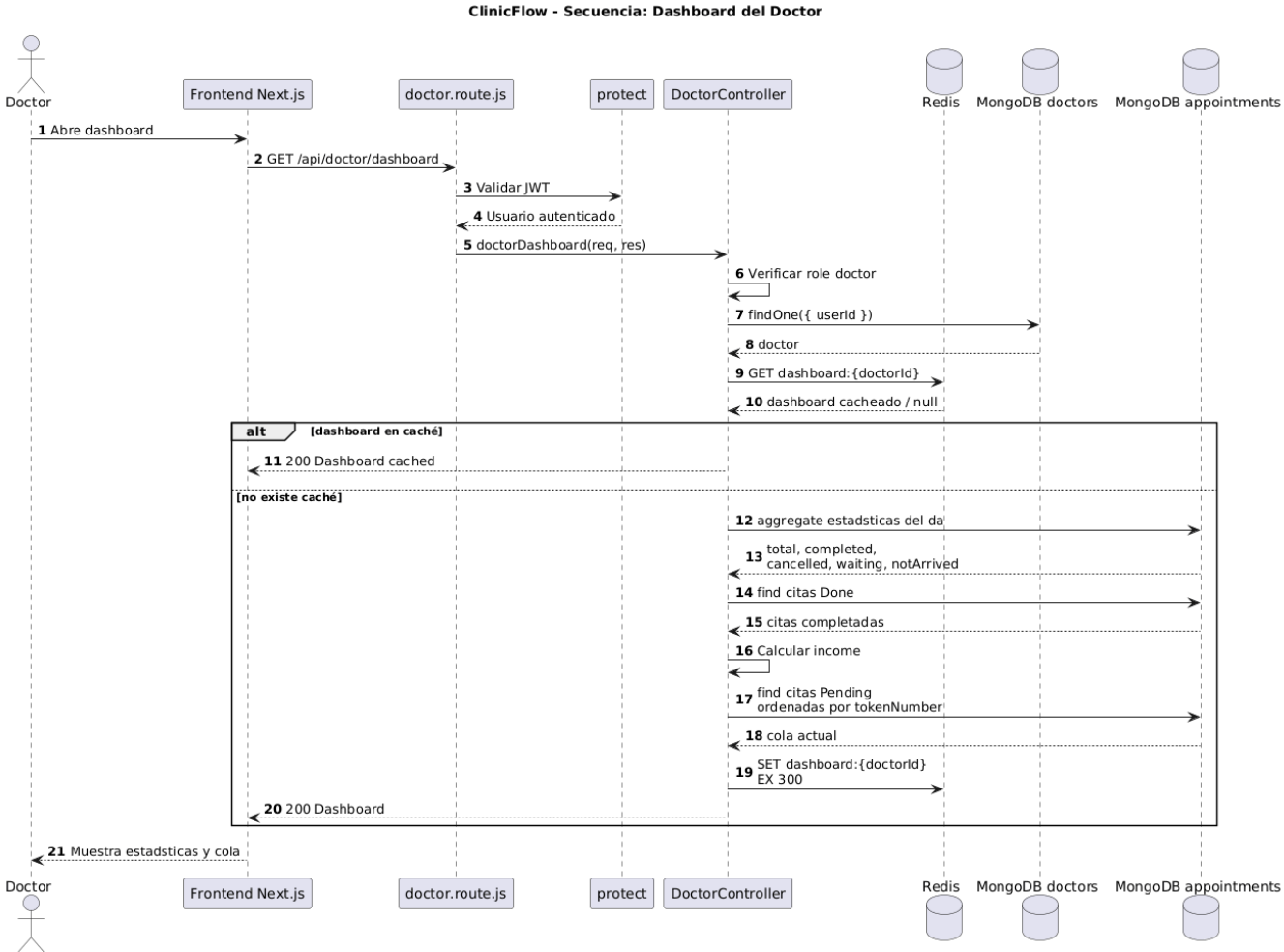


Figura 11. Diagrama de secuencia: Llamar siguiente paciente

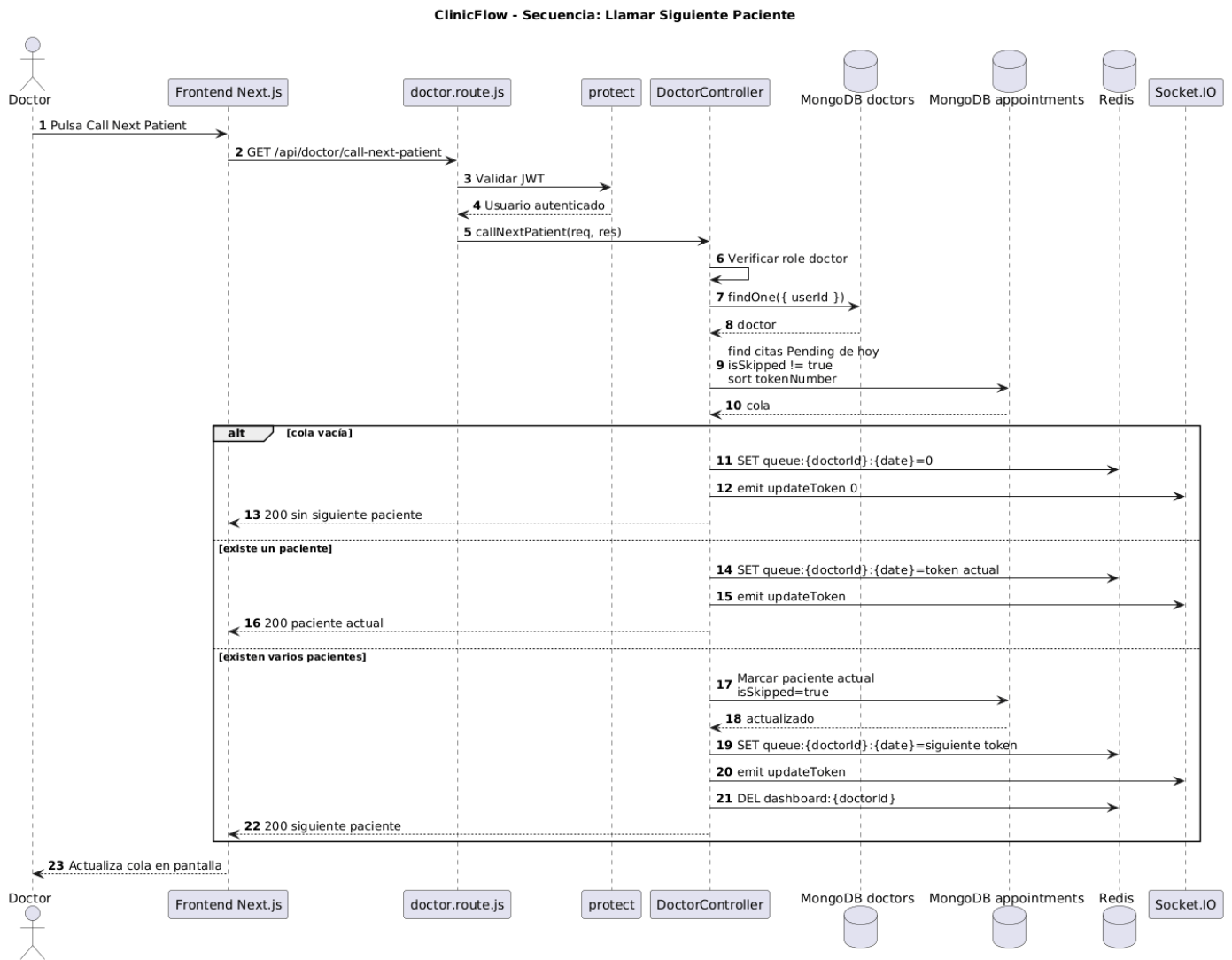
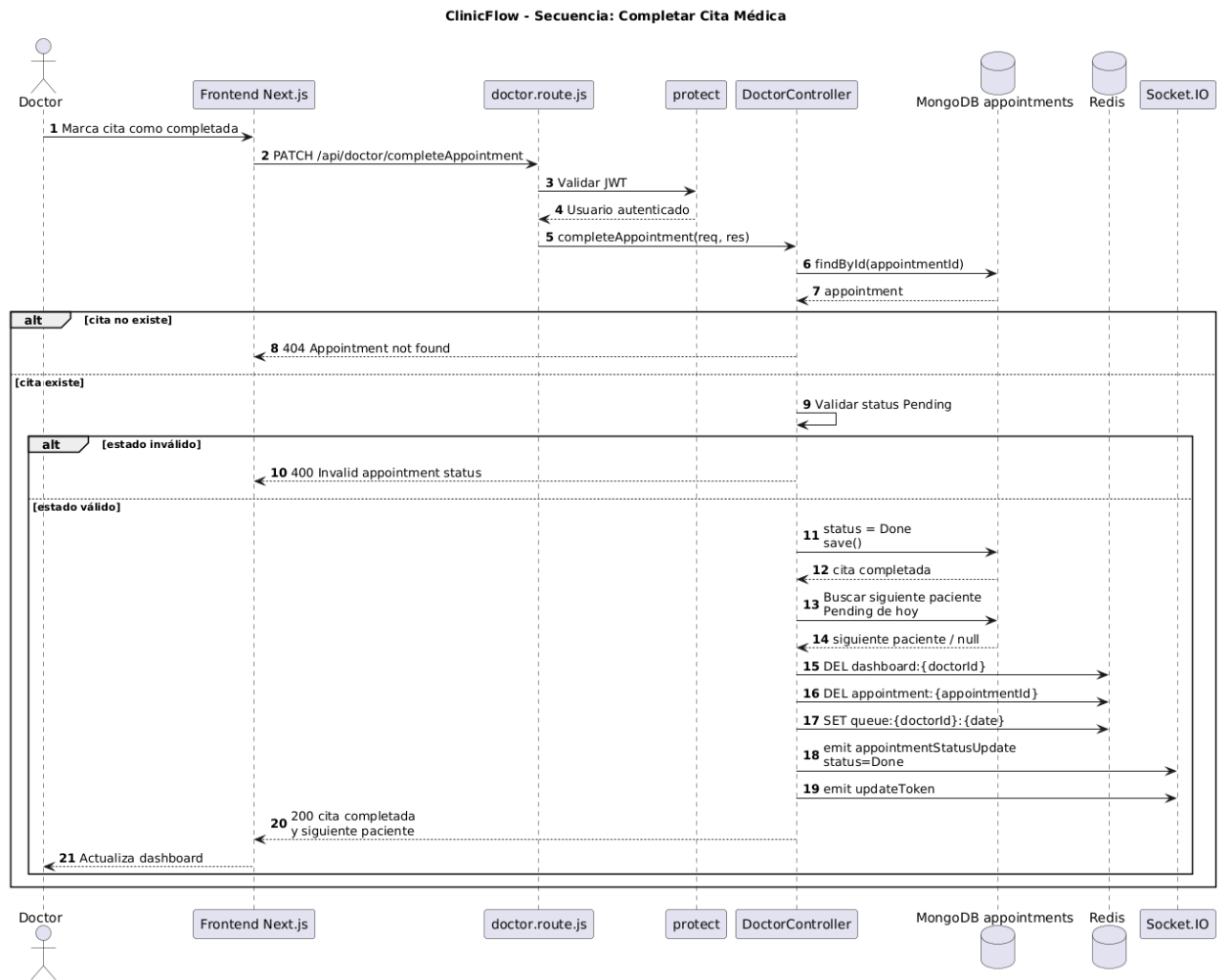


Figura 12. Diagrama de secuencia: completar cita médica



7. CONCLUSIONES Y RECOMENDACIONES:

7.1. Conclusiones

- La ingeniería inversa permitió comprender la estructura interna de ClinicFlow sin necesidad de ejecutar el sistema, mediante el análisis del código fuente, rutas, controladores, modelos y componentes.
- Se identificó que la aplicación está organizada bajo una arquitectura web cliente-servidor, con frontend en Next.js, backend en Express, MongoDB como base de datos, Redis como caché y Socket.IO para comunicación en tiempo real.
- Los diagramas generados permiten representar diferentes vistas del sistema: estructura de clases, componentes de arquitectura, diseño de base de datos y comportamiento temporal de los procesos implementados.

- La documentación obtenida sirve como base para continuar con actividades posteriores de mantenimiento, tales como identificación de problemas, priorización, implementación de cambios y pruebas.

7.2. Recomendaciones

- Mantener actualizados los diagramas cuando se realicen cambios en el código fuente, para evitar que la documentación quede desfasada.
- Complementar el análisis estático con una ejecución controlada de la aplicación en prácticas posteriores, con el fin de validar los flujos modelados.
- Separar claramente los diagramas estructurales de los diagramas de comportamiento para facilitar la comprensión del sistema.
- Incluir dentro del archivo comprimido final el informe, el sistema analizado y todos los diagramas generados con nombres consistentes.

8. BIBLIOGRAFÍA:

- Pressman, R. S. Ingeniería del software: un enfoque práctico.
- Sommerville, I. Ingeniería de software.
- IEEE. Software Maintenance standards and software engineering practices.
- Lucidchart. Guías de diagramas UML y diagramas entidad-relación.
- Repositorio ClinicFlow. Código fuente analizado para la práctica de ingeniería inversa.

.